

Phoneme Pronunciation Practice App

Project handoff summary — built with Claude, continuing in Cursor

Project Overview

A browser-based tool for students to practice pronouncing English phoneme groups. The student sees a nonsense or real word, speaks it aloud, and receives layered feedback: speech-to-text word matching, DSP-based spectral analysis, and an online-learning neural network that personalizes to their voice over the session.

What Has Been Built

Phoneme groups. Five selectable groups: *bl/cl/fl* onset clusters, *str/spr/scr* sibilant clusters, short vowels (/ae/, /e/, /i/, /o/, /u/), long vowels (/ei/, /i:/, /ai/, /ou/, /ju:/), and ending blends (-mp, -nt, -sk, -ft, -nd). Each group has 10 practice words.

Real-time waveform. Web Audio API `AnalyserNode` feeds a canvas waveform during recording. `MediaRecorder` captures audio chunks simultaneously.

DSP pipeline (pure JS). Implemented from scratch: Cooley-Tukey FFT (512-point), 26-filter mel filterbank (80 Hz–Nyquist), log mel energies, DCT-II producing 13 MFCCs. Also computes ZCR, RMS, and spectral centroid per frame. No external libraries.

MFCC heatmap. Canvas-rendered blue-white-red heatmap after each recording. X-axis = time frames (~5 ms each), Y-axis = coefficients *c0*–*c12*.

Heuristic feedback. Group-specific rules: spectral centroid and ZCR for onset detection (/s/ needs centroid >3800 Hz, /f/ needs >3200 Hz, stops need <2400 Hz); MFCC[1] and MFCC[2] as rough F1/F2 proxies for vowel quality; RMS at coda for ending blends.

Speech-to-text matching. Web Speech API (`SpeechRecognition`) transcribes each attempt. Match logic: exact string, substring, or Levenshtein distance ≤ 1 .

Neural network (pure JS). 13-input → 16 (ReLU) → 8 (ReLU) → 6-class softmax. Glorot uniform initialization. Cross-entropy loss. Backprop + batch SGD. Trains online: each correct vowel attempt adds a (MFCC nucleus, vowel class) sample. Predictions shown as probability bars; labeled "cold" until ≥ 8 samples.

Session scoring. Correct / total / accuracy tracked. Last 5 attempts shown in history with word, transcription, and pass/fail badge.

Architecture & File Structure

The project is currently a single self-contained HTML file. For Cursor development, the recommended structure is:

```

phoneme-app/ index.html – shell, imports src/ dsp.js – FFT, mel filterbank, DCT,
extractFrames() net.js – Net class (forward, backprop, train, predict) recorder.js –
MediaRecorder + Web Audio setup asr.js – SpeechRecognition wrapper feedback.js –
heuristic rules per phoneme group ui.js – DOM updates, heatmap, prob bars, history
data.js – GROUPS, VC, W2C, VH constants styles/ main.css

```

DSP Pipeline Detail

Constants:

```

FSIZE = 512 // FFT frame size (samples) HSIZE = 256 // hop size → 50% overlap NMEL =
26 // mel filterbank filters NMCC = 13 // MFCC coefficients kept

```

Per frame (called inside extractFrames(audio, sampleRate)):

1. Slice 512 samples, apply Hamming window
2. Cooley-Tukey FFT → magnitude spectrum (256 bins)
3. Apply 26 triangular mel filters (80 Hz to sr/2)
4. Take log of each filter energy
5. DCT-II of 26 log-mel values → keep first 13 as MFCCs
6. Compute ZCR (zero-crossing rate) and RMS for the frame
7. Compute spectral centroid: $\text{sum}(\text{freq} * \text{mag}) / \text{sum}(\text{mag})$

Vowel nucleus extraction (for neural net input):

Filter frames by RMS > 12% of peak (voiced region). Take the middle 60% of voiced frames as the nucleus. Average the 13 MFCCs across nucleus frames → 13-dim feature vector → net input.

Neural Network Detail

Architecture:

```

Input 13 – MFCC nucleus averages Hidden 16 – ReLU Hidden 8 – ReLU Output 6 – Softmax
(vowel classes)

```

6 vowel classes (W2C mapping):

Index	Label	IPA	Training words
0	high-front	/i/ /e/	feet, seed, bit, tip
1	mid-front	/e/ /æ/	lake, bake, bet, peg, hem
2	low-front	/æ/	bat, cap, fan
3	low-back	/a/ /o/ /ɑ/	bot, dog, bike, kite
4	high-back	/u/ /ju/	cute, fuse

5	mid-central	/■/ /o■/	but, cup, coat, roam
---	-------------	----------	----------------------

Training loop:

Student speaks a word → SpeechRecognition returns transcript
 If transcript matches target word (exact / substring / Levenshtein ≤ 1): correct
 If correct AND word is in W2C AND lastNuc is not null:
 push (nucMFCC, oneHot(classIndex, 6)) into training buffer TD
 call net.train(TD.X, TD.Y) — full-batch SGD over all accumulated data
 update status bar with new loss value

Backprop (implemented from scratch):

Output delta: pred - y (softmax + cross-entropy identity). Hidden layer delta: flow delta through weight transpose, mask by ReLU derivative (1 if $z > 0$, else 0). Weight update: $W -= lr * (a_{prev}^T @ \text{delta}) / \text{batch_size}$. Learning rate: 0.02.

Known Issues & Next Steps

Widget sandbox issue. The artifact iframe in Claude.ai appears to block script execution in some configurations. The app works correctly when served from a local dev server (Vite, npx serve, etc.).

Neural net starts cold. Glorot-uniform initialization is random. Predictions are meaningless until ~8-10 correct vowel attempts have been collected. Consider shipping with pre-trained weights derived from a small labeled corpus, or seed with phonetically-informed biases.

Vowel class coverage uneven. high-back (/u■/) only has 2 training words (cute, fuse). Long vowel group needs more words per class for reliable convergence.

Consonant groups use heuristics only. The neural net only runs on vowel groups. Onset and coda groups rely entirely on spectral centroid and ZCR thresholds, which are approximate.

Recommended next: serve locally. Run with Vite. Split into modules (dsp.js, net.js, etc.). Add a pre-training pass using the Web Audio offline context on reference recordings.

Recommended next: better model. Replace the hand-rolled net with TensorFlow.js (served locally, WASM backend works fine outside the sandbox). Consider loading a pre-trained phoneme embedding or fine-tuning on user recordings across sessions via localStorage.

Recommended next: more feedback modes. Show formant plot (F1 vs F2 vowel space). Add DTW comparison against a reference recording. Add per-phoneme accuracy tracking over multiple sessions.

Browser API Reference

navigator.mediaDevices.getUserMedia — Microphone access

MediaRecorder — Captures audio as webm/opus chunks; onstop triggers processAudio()

AudioContext / createMediaStreamSource — Routes mic stream into Web Audio graph

AnalyserNode (fftSize=2048) — Real-time waveform data via `getByteTimeDomainData()`

audioCtx.decodeAudioData() — Decodes webm blob to PCM Float32Array for DSP

SpeechRecognition (webkitSpeechRecognition) — Word-level transcription; Chrome/Edge only

OfflineAudioContext — Not currently used — useful for reference recording analysis

Generated from a Claude conversation session. Speech recognition requires Chrome or Edge. Serve over HTTPS or localhost for microphone access.